Developing Full Stack Next.js Web Applications

Dr. Jose Annunziato

Table of Contents

Chapter 3 - Creating Single Page Applications with React

JavaScript, officially known as ***ECMAScript***, is a programming language most famous for writing scripts that run in and control web browsers. The name "***ECMAScript***" originates from the ***European Computer Manufacturers Association*** (***ECMA***), which standardized the language in 1997 to ensure consistency across implementations. While "***JavaScript***" is the popular name used by developers, ***ECMAScript*** is its formal designation, reflecting its roots in the standardization process. Over the years, ***ECMAScript*** has evolved significantly, with a major milestone in 2015 when ***ECMAScript 2015*** (commonly referred to as ***ES6***) was released. This version introduced transformative features like ***arrow functions***, ***classes***, ***let***/***const*** declarations, and ***modules***, making JavaScript more robust and suitable for modern application development. These advancements laid the groundwork for libraries like ***React***, which simplifies building ***Single Page Applications*** (***SPA***s). While ***JavaScript*** was historically used for creating static web pages, this chapter explores how to leverage it for handling user input, manipulating data structures, parameterizing components, and creating data-driven user interfaces with React.

TypeScript, developed by ***Microsoft*** and first released in 2012, is a superset of JavaScript that adds ***static typing*** and advanced tooling to enhance code ***reliability*** and ***scalability***. Designed to address the challenges of large-scale JavaScript applications, TypeScript introduces features like ***interfaces***, ***type annotations***, and ***compile-time type checking***, which help developers catch errors early and improve ***maintainability***. As an open-source language, TypeScript compiles to plain JavaScript, ensuring compatibility with existing JavaScript environments, including web browsers and Node.js. Its adoption has grown rapidly, particularly in frameworks like React and Angular, making it a powerful tool for building robust, data-driven applications in this course.

3.1 Learning Objectives

By the end of this chapter, you will be able to:

- Understand the basics of JavaScript and its role in Web development
- Learn about **JavaScript variables, constants, data types** and their applications
- Work with **Boolean variables and conditionals**
- Use the **ternary conditional operator** to generate conditional output
- Define and utilize **JavaScript functions**, including arrow functions and implied returns
- Implement **template string literals** for efficient string handling
- Manipulate **JavaScript data structures**, including arrays and objects
- Utilize **array functions** such as `map()`, `find()`, `filter()`, and `findIndex()`
- Convert data using **JSON (JavaScript Object Notation)** and `JSON.stringify()`
- Learn and apply **the spread operator** and destructuring in JavaScript
- Use **dynamic styling** in React applications with HTML classes and style attributes
- Parameterize React components using **child components** and **location-based data**
- Implement a **data-driven Kambaz application**
- Work with **data-driven navigation, dashboards, and courses**
- Develop **data-driven modules and assignments screens**
- Understand the structure of a **single-page application (SPA)** using React.

3.2 Introduction to JavaScript

To achieve the learning objectives of understanding JavaScript fundamentals and their application in React for building dynamic SPAs, we'll begin with hands-on exercises that cover core concepts like variables, functions, data structures, and more.

In the following exercises, we'll learn about the ***JavaScript*** programming language. We'll create a component for each exercise and import them into the ***Lab3*** page component. Create ***app/labs/lab3/page.tsx***, import it from the ***Labs*** page, and create a link to the new ***Lab3*** page. Also add a link to the new ***Lab3*** page in the ***TOC.tsx***. Confirm that you can navigate to the ***Lab3*** page from the ***Labs*** page and from ***TOC.tsx***.

app/labs/lab3/page.tsx

```
export default function Lab3() {
  return (
    <div id="wd-lab3">
      <h3>Lab 3</h3>
    </div>
);}
```

3.2.1 Variables and Constants

Variables can store state information about applications such as user information, preferences, courses, and enrollments. To practice declaring variables and constants, create the ***VariablesAndConstants*** component below and import it from the ***Lab3*** component. Confirm the browser displays as shown in Figure 3.2.1. We'll be creating several components to practice various features of the ***JavaScript*** language. Import them into the ***Lab3*** page component and confirm the output is as described for each of the lab exercises.

app/labs/lab3/VariablesAndConstants.tsx

```
export default function VariablesAndConstants() {
 var functionScoped = 2;
 let blockScoped = 5;
 const constant1 = functionScoped - blockScoped;
 return(
   <div id="wd-variables-and-constants">
     <h4>Variables and Constants</h4>
     functionScoped = { functionScoped }<br/>
     blockScoped = { blockScoped }<br/>
     constant1 = { constant1 }<hr/>
   </div>
);}
```

app/labs/lab3/page.tsx

```
import VariablesAndConstants
  from "./VariablesAndConstants";

export default function Lab3() {
  return(
    <div id="wd-lab3">
      <h3>Lab 3</h3>
      <VariablesAndConstants/>
    </div>
  );
}
```

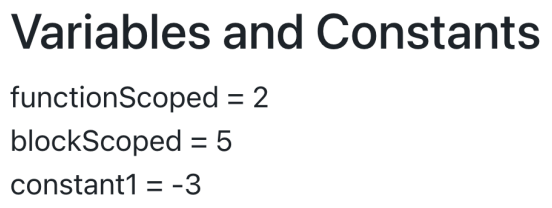


Figure 3.2.1 - Variables and Constants

3.2.2 Data Types

JavaScript declares several datatypes such as ***Number***, ***String***, ***Date***, and so on. To practice with variable types, create the ***VariableTypes*** component shown below and import it at the bottom of the ***Lab3*** component. Confirm that the browser renders as shown in Figure 3.2.2. Note that we had to convert the boolean variable into a string

type before it could render in the browser. In React, boolean values like true or false are not rendered directly in JSX, so concatenating a boolean with an empty string (e.g., ***booleanVariable + ""***) converts it to a string representation, ensuring it displays correctly in the UI.

app/labs/lab3/VariableTypes.tsx

```
export default function VariableTypes() {
  let numberVariable = 123;
  let floatingPointNumber = 234.345;
  let stringVariable = 'Hello World!';
  let booleanVariable = true;
  let isNumber = typeof numberVariable;
  let isString = typeof stringVariable;
  let isBoolean = typeof booleanVariable;
  return(
    <div id="wd-variable-types">
      <h4>Variables Types</h4>
      numberVariable = { numberVariable }<br/>
      floatingPointNumber = { floatingPointNumber }<br/>
      stringVariable = { stringVariable }<br/>
      booleanVariable = { booleanVariable + "" }<br/>
      isNumber = { isNumber }<br/>
      isString = { isString }<br/>
      isBoolean = { isBoolean }<hr/>
    </div>
);}
```

3.2.3 Boolean Variables

To practice with ***Boolean*** data types, create a component called ***BooleanVariables*** and import it in the ***Lab3*** component. Use the previous lab exercises as a guide of how to complete this exercise. The new component should add a new section called ***Boolean Variables*** that displays each of the new variables so that the browser renders as shown in Figure 3.2.3. You might need to cast the boolean values to string by concatenating an empty string to display the variables, e.g., ***false3 = {false3 + ""}
. Add ***function, ***export***, and other keywords as needed.

app/labs/lab3/BooleanVariables.tsx

```
let numberVariable = 123, floatingPointNumber = 234.345;
let true1 = true, false1 = false;
let false2 = true1 && false1;
let true2 = true1 || false1;
let true3 = !false2;
let true4 = numberVariable === 123; // always use === not ==
let true5 = floatingPointNumber !== 321.432;
let false3 = numberVariable < 100;
return (
  <div id="wd-boolean-variables">
    <h4>Boolean Variables</h4>
    true1     = {true1 + ""}     <br />
    false1    = {false1 + ""}    <br />
    false2    = {false2 + ""}    <br />
    true2     = {true2 + ""}     <br />
    true3     = {true3 + ""}     <br />
    true4     = {true4 + ""}     <br />
    true5     = {true5 + ""}     <br />
    false3    = {false3 + ""}    <hr />
  </div>
);
```

Variables Types

numberVariable = 123
floatingPointNumber = 234.345
stringVariable = Hello World!
booleanVariable = true
isNumber = number
isString = string
isBoolean = boolean

Figure 3.2.2 - Data Types

Boolean Variables

true1 = true
false1 = false
false2 = false
true2 = true
true3 = true
true4 = true
true5 = true
false3 = false

Figure 3.2.3 - Boolean Variables

3.2.4 Conditionals

Conditional expressions allow scripts to make decisions based on predicates that compare values and variables. Scripts can decide to execute different parts of the code based on the result of these predicates using ***if/else*** and other constructs. Create the following components and import them into the ***Lab3*** component. Confirm that the components render as shown below. The most common use of conditionals is ***if/else*** statements that evaluate a predicate and can decide to execute one of two different code blocks depending on whether the predicate evaluates to ***true*** or ***false***. To practice with ***if/else***, create a component called ***IfElse*** based on the code shown below. It should render a new section labeled ***If Else*** and render as shown in Figure 3.2.4. The ***true1*** paragraph is only rendered if ***true1*** is true. The ***ternary operators*** ? and : can be used to render one of two options based on the value of a boolean expression.

app/labs/lab3/IfElse.tsx

```
let true1 = true, false1 = false;
return (
   <div id="wd-if-else">
      <h4>If Else</h4>
      { true1 && <p>true1</p> }
      { !false1 ? <p>!false1</p> : <p>false1</p> } <hr/>
   </div>
);
```

3.2.5 Ternary Conditional Operator

Ternary conditional operators are concise alternative to ***if/else*** statements. It takes three arguments

1. A ***predicate*** expression that evaluates to true or false followed by a question mark (?)
2. An expression that evaluates if the ***predicate*** is ***true*** followed by a colon (:)
3. Followed by an expression that evaluates iff the ***predicate*** is ***false***

To practice the ternary operator, create a new component called ***TernaryOperator*** based on the code shown below which should render as shown in Figure 3.2.5.

app/labs/lab3/TernaryOperator.tsx

```
let loggedIn = true;
return(
   <div id="wd-ternary-operator">
      <h4>Logged In</h4>
      { loggedIn ? <p>Welcome</p> : <p>Please login</p> } <hr/>
   </div>
)
```

3.2.6 Generating conditional output

With boolean expressions we can render content based on some logic. The following example decides rendering one content versus another based on a simple boolean constant ***loggedIn***. If a user is ***loggedIn***, then the component renders a greeting, otherwise suggests the user should login. Implement the following component to practice conditional rendering.

app/labs/lab3/ConditionalOutputIfElse.tsx

```
export default function ConditionalOutputIfElse() {
 const loggedIn = true;
 if (loggedIn) {
   return <h2 id="wd-conditional-output-if-else-welcome">Welcome If Else</h2>;
 } else {
   return (
     <h2 id="wd-conditional-output-if-else-login">Please login If Else</h2>
   );
}}
```

A more compact way we can achieve the same thing is by including the conditional content in a boolean expression that short circuits the content if its false, or evaluates the expression if it's true. Implement the equivalent component shown in Figure 3.2.6.

app/labs/lab3/ConditionalOutputInline.tsx

```
export default function ConditionalOutputInline() {
 const loggedIn = false;
 return (
   <div id="wd-conditional-output-inline">
     {loggedIn && <h2>Welcome Inline</h2>}
     {!loggedIn && <h2>Please login Inline</h2>}
   </div>
);}
```

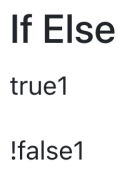


Figure 3.2.4 - Conditionals

Figure 3.2.5 - Ternary Conditional Operator

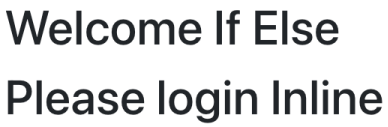


Figure 3.2.6 - Generating conditional output

3.3 JavaScript Functions

Functions allow reusing an algorithm by wrapping it in a named, parameterized code block. ***JavaScript*** supports two styles of functions based on the language history. Functions are declared using the following syntax.

```
function <functionName> (<parameterList>) { <functionBody> }
```

To practice using functions create a new component called ***LegacyFunctions*** based on the code below. Import this new component in the ***Lab3*** component and confirm the browser renders as shown in Figure 3.3.

app/Labs/Lab3/LegacyFunctions.tsx

```
function add(a: number, b: number) {
  return a + b;
}
export default function LegacyFunctions() {
  const twoPlusFour = add(2, 4);
  console.log(twoPlusFour);
  return (
    <div id="wd-legacy-functions">
      <h4>Functions</h4>
      <h5>Legacy ES5 functions</h5>
      twoPlusFour = {twoPlusFour}    <br />
      add(2, 4) = {add(2, 4)}        <hr />
    </div>
);}
```

3.3.1 Arrow functions

A new version of ***JavaScript*** was introduced in 2015 and is officially referred to as ***ECMAScript 6*** or ***ES6***. A new syntax for declaring functions was introduced which is less verbose and provides tons of new features we'll explore throughout this course. This function syntax is often referred to as ***arrow functions***. To practice using ES6 arrow functions, create a new component called ***ArrowFunctions*** based on the code below. Import this new component in the ***Lab3*** component and confirm the browser renders as shown in Figure 3.3.1.

app/Labs/Lab3/ArrowFunctions.tsx

```
const subtract = (a: number, b: number) => {
  return a - b;
};

export default function ArrowFunctions() {
  const threeMinusOne = subtract(3, 1);
  console.log(threeMinusOne);
  return (
    <div id="wd-arrow-functions">
      <h4>New ES6 arrow functions</h4>
      threeMinusOne = {threeMinusOne}   <br />
      subtract(3, 1) = {subtract(3, 1)} <hr />
    </div>
  );
}
```

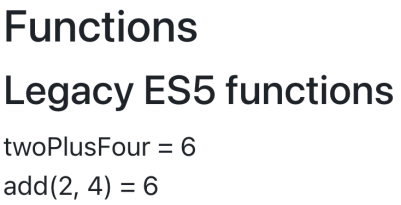


Figure 3.3 - JavaScript Functions

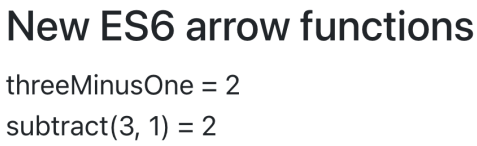


Figure 3.3.1 - Arrow functions

NOTE: Throughout the last couple of exercises we've provided code in the return statement to render the variables in the browser and asked that you confirm the output matches. Going forward we'll omit the return statement, but continue to implement it and confirm the output matches the one provided.

3.3.2 Implied Returns

One of the new features of the new ES6 functions is ***implied returns***, that is, if the body of the function consists of just returning some value or expression, then the return statement is optional and can be replaced with just the value or expression. To practice this feature create a new component called ***ImpliedReturn*** based on the code below. Import this new component in the ***Lab3*** component and confirm the browser renders as shown in Figure 3.3.2.

app/labs/lab3/ImpliedReturn.tsx

```
const multiply = (a: number, b: number) => a * b;
const fourTimesFive = multiply(4, 5);
console.log(fourTimesFive);

return (
  <div id="wd-implied-return">
    <h4>Implied return</h4>
    fourTimesFive = {fourTimesFive}   <br />
    multiply(4, 5) = {multiply(4, 5)} <hr />
  </div>
);
```

3.3.3 Template String Literals

Generating dynamic HTML consists of writing code that manipulates and concatenates strings to generate new HTML strings based on some program logic. Basically consists of one language writing code in another language, similar to what a compiler does. Working with strings can be error prone especially if you have to use lots of extra operations and variables to concatenate the resulting string. JavaScript template strings provide a better approach by allowing embedding expressions and algorithms right within strings themselves. To practice, implement a new component called ***TemplateLiterals*** based on the code below. Import this new component in ***Lab3*** and confirm the browser renders as shown. In your return statement, wrap the HTML output in a ***DIV*** whose ***ID*** is ***wd-template-literals***. Do not hard code the results ***5***, ***Welcome home alice***, etc. Instead use the values of variables ***result1***, ***result2***, etc., to render the the output shown in Figure 3.3.3.

app/labs/lab3/TemplateLiterals.tsx

```
export default function TemplateLiterals() {
  const five = 2 + 3;
  const result1 = "2 + 3 = " + five;
  const result2 = `2 + 3 = ${2 + 3}`;
  const username = "alice";
  const greeting1 = `Welcome home ${username}`;
  const loggedIn = false;
  const greeting2 = `Logged in: ${loggedIn ? "Yes" : "No"}`;
  return (
    <div id="wd-template-literals">
      <h4>Template Literals</h4>
      result1 = {result1}     <br />
      result2 = {result2}     <br />
      greeting1 = {greeting1} <br />
      greeting2 = {greeting2} <hr />
    </div>
  );
}
```

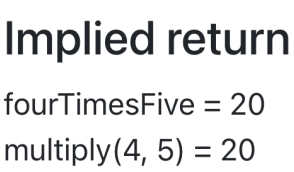


Figure 3.3.2 - Implied Returns

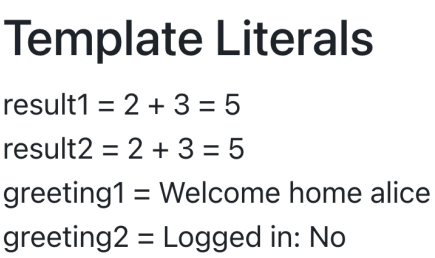


Figure 3.3.3 - Template String Literals

3.4 JavaScript Data Structures

Up to this point we have been discussing ***primitive datatypes*** such as ***strings***, ***numbers***, and ***booleans***. These can be combined into ***complex datatypes*** such as ***arrays***, ***objects***, and ***classes***. Arrays can group together several values into a single variable. Arrays can group together values of different datatypes, e.g., number arrays, string arrays, and even a mix and match of datatypes in the same array. Not that you would ever want to actually do that. To practice with arrays create a component called ***SimpleArrays*** and copy and paste the code below. Import the component to the ***Lab3*** component and confirm the browser renders as shown in Figure 3.4. Note that the arrays render without the commas. This feature will come in handy when the array items are ***HTML*** elements.

app/labs/lab3/SimpleArrays.tsx

```
export default function SimpleArrays() {
  var functionScoped = 2;  let blockScoped = 5;
  const constant1 = functionScoped - blockScoped;
  let numberArray1 = [1, 2, 3, 4, 5];
  let stringArray1 = ["string1", "string2"];
  let htmlArray1 = [<li key={1}>Buy milk</li>, <li key={2}>Feed the pets</li>];
  let variableArray1 = [ functionScoped, blockScoped, constant1,
                         numberArray1, stringArray1 ];
  return (
    <div id="wd-simple-arrays">
      <h4>Simple Arrays</h4>
      numberArray1 = {numberArray1}     <br />
      stringArray1 = {stringArray1}     <br />
      variableArray1 = {variableArray1} <br />
      Todo list:
      <ol>{htmlArray1}</ol>
      <hr />
    </div> );}
```

3.4.1 Array index and length

An array's length is available as property ***length***. The ***indexOf()*** function allows finding where a particular array member is found. To practice with array indices and length, implement a new component called ***ArrayIndexAndLength*** based on the code below. Import this new component in ***Lab3*** and confirm the browser renders as shown in Figure 3.4.1.

app/labs/lab3/ArrayIndexAndLength.tsx

```
export default function ArrayIndexAndLength() {
  let numberArray1 = [1, 2, 3, 4, 5];
  const length1 = numberArray1.length;
  const index1 = numberArray1.indexOf(3);
  return (
    <div id="wd-array-index-and-length">
      <h4>Array index and length</h4>
```

```
      length1 = {length1} <br />
      index1 = {index1}   <hr />
    </div>
);}
```

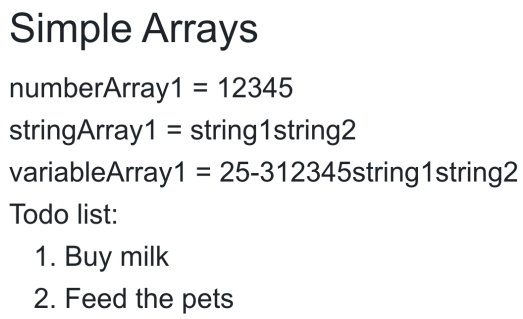


Figure 3.4 - JavaScript Data Structures

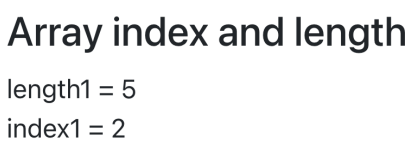


Figure 3.4.1 - Array index and length

3.4.2 Adding and Removing Data to/from Arrays

In most languages arrays are immutable, whereas in JavaScript elements can easily be added and removed from arrays. The ***push()*** function appends elements at the end of an array. The ***splice()*** function removes/adds elements anywhere in the array. To practice adding/removing data from arrays, implement component ***AddingAndRemovingDataToFromArrays*** based on the code below. Import this new component in ***Lab3*** and confirm the browser renders as shown in Figure 3.4.2.

app/labs/lab3/AddingAndRemovingToFromArrays.tsx

```
export default function AddingAndRemovingToFromArrays() {
  let numberArray1 = [1, 2, 3, 4, 5];
  let stringArray1 = ["string1", "string2"];
  let todoArray = [<li>Buy milk</li>, <li>Feed the pets</li>];
  numberArray1.push(6); // adding new items
  stringArray1.push("string3");
  todoArray.push(<li>Walk the dogs</li>);
  numberArray1.splice(2, 1); // remove 1 item starting at 2
  stringArray1.splice(1, 1);
  return (
    <div id="wd-adding-removing-from-arrays">
      <h4>Add/remove to/from arrays</h4>
      numberArray1 = {numberArray1} <br />
      stringArray1 = {stringArray1} <br />
      Todo list:
      <ol>{todoArray}</ol><hr />
    </div> );}
```

3.4.3 For Loops

We can operate on each array value by iterating over them in a ***for loop***. To practice with for loops, implement a new component called ***ForLoops*** based on the code below. Import this new component in ***Lab3*** and confirm the browser renders as shown in Figure 3.4.3.

app/labs/lab3/ForLoops.tsx

```
export default function ForLoops() {
  let stringArray1 = ["string1", "string3"];
  let stringArray2 = [];
  for (let i = 0; i < stringArray1.length; i++) {
```

```
    const string1 = stringArray1[i];
    stringArray2.push(string1.toUpperCase());
  }
  return (
    <div id="wd-for-loops">
      <h4>Looping through arrays</h4>
      stringArray2 = {stringArray2} <hr />
    </div> );}
```

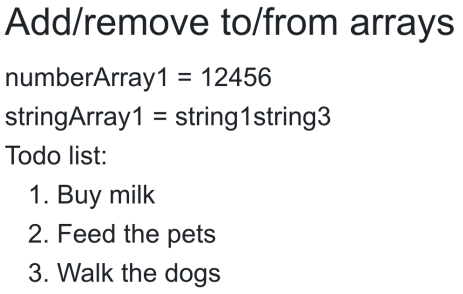


Figure 3.4.2 - Adding and Removing Data to/from Arrays

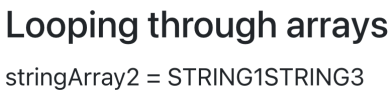


Figure 3.4.3 - For Loops

3.4.4 The Map Function

An array's ***map*** function can iterate over an array's values, apply a function to each value, and collate all the results in a new array. The first example below iterates over the ***numberArray1*** and calls the ***square*** function for each element. The ***square*** function was declared earlier in this document and it accepts a parameter and returns the square of the parameter. The ***map*** function collates all the squares into a new array called ***squares*** as shown below. The second example does the same thing, but uses a function that calculates the ***cubes*** of all numbers in the same ***numberArray1*** array. To practice with ***map***, implement a new component called ***MapFunction*** based on the code below. Import this new component in ***page.tsx*** and confirm the browser renders as shown in Figure 3.4.4.

app/labs/lab3/MapFunction.tsx

```
export default function MapFunction() {
  let numberArray1 = [1, 2, 3, 4, 5, 6];
  const square = (a: number) => a * a;
  const todos = ["Buy milk", "Feed the pets"];
  const squares = numberArray1.map(square);
  const cubes = numberArray1.map((a) => a * a * a);
  return (
    <div id="wd-map-function">
      <h4>Map Function</h4>
      squares = {squares} <br />
      cubes = {cubes} <br />
      Todos:
      <ol>
        {todos.map((todo) => (
          <li>{todo}</li>
        ))}
      </ol> <hr/>
    </div>
  );
}
```

3.4.5 The Find Function

An array's ***find*** function can search for an item in an array and return the element it finds. The find function takes a function as an argument that serves as a predicate. The predicate should return true if the element is the one

you're looking for. The predicate function is invoked for each of the elements in the array and when the function returns true, the find function stops because it has found the element that it was looking for. To practice, implement a new component called ***FindFunction*** based on the code below. Import this new component in ***JavaScript*** and confirm the browser renders as shown in Figure 3.4.5.

app/labs/lab3/FindFunction.tsx

```
export default function FindFunction() {
  let numberArray1 = [1, 2, 3, 4, 5];
  let stringArray1 = ["string1", "string2", "string3"];
  const four = numberArray1.find((a) => a === 4);
  const string3 = stringArray1.find((a) => a === "string3");
  return (
    <div id="wd-find-function">
      <h4>Find Function</h4>
      four = {four} <br />
      string3 = {string3} <hr />
    </div>
);}
```

3.4.6 The Find Index Function

Alternatively we can use ***findIndex*** function to determine the index where an element is located inside an array. Copy the code and display the content as shown in Figure 3.4.6.

app/labs/lab3/FindIndex.tsx

```
let numberArray1 = [1, 2, 4, 5, 6];
let stringArray1 = ['string1', 'string3'];

const fourIndex = numberArray1.findIndex(a => a === 4);
const string3Index = stringArray1.findIndex(a => a === 'string3');
```

3.4.7 The Filter Function

The ***filter*** function can look for elements that meet a criteria and collate them into a new array. For instance, the example below is looking through the ***numberArray1*** array for all values that are greater than 2. Then we look for all even numbers and then for all odd numbers. All the results are stored in corresponding arrays with appropriate names. To practice, implement a new component called ***FilterFunction*** based on the code below. Import this new component in ***Lab3*** and confirm the browser renders as shown in Figure 3.4.7.

app/labs/lab3/FilterFunction.tsx

```
export default function FilterFunction() {
  let numberArray1 = [1, 2, 4, 5, 6];
  const numbersGreaterThan2 = numberArray1.filter((a) => a > 2);
  const evenNumbers = numberArray1.filter((a) => a % 2 === 0);
  const oddNumbers = numberArray1.filter((a) => a % 2 !== 0);
  return (
    <div id="wd-filter-function">
      <h4>Filter Function</h4>
      numbersGreaterThan2 = {numbersGreaterThan2}  <br />
      evenNumbers = {evenNumbers}    <br />
      oddNumbers = {oddNumbers}      <hr />
    </div>
);}
```

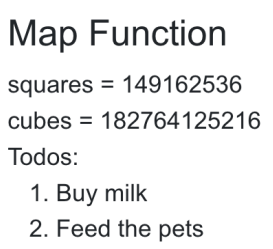


Figure 3.4.4 - The Map Function

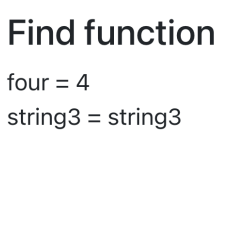


Figure 3.4.5 - The Find Function

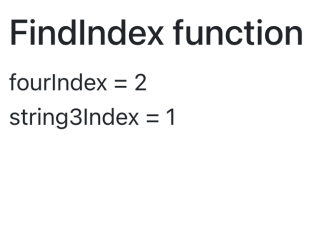


Figure 3.4.6 - The Find Index Function

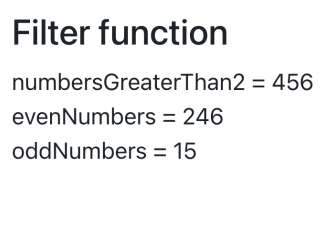


Figure 3.4.7 - The Filter Function

3.4.8 JSON Stringify

JavaScript has a global object called ***JSON*** which stands for ***JavaScript Object Notation***. The object provides several useful formatting functions such as ***stringify()*** and ***parse()***. Stringify converts ***JavaScript*** data structures to formatted strings. For instance let's format the following arrays and display them in the browser. Note how the array is rendered with square brackets and items are separated by commas as shown in Figure 3.4.8.

app/labs/lab3/JsonStringify.tsx

```
export default function JsonStringify() {
  const squares = [1, 4, 16, 25, 36];
  return (
    <div className="wd-json-stringify">
      <h3>JSON Stringify</h3>
      squares = {JSON.stringify(squares)}
      <hr />
    </div>
);}
```

3.4.9 JavaScript Object Notation (JSON)

Multiple values, of various datatypes can be combined together to create complex datatypes called ***objects***. For example the code below declares a ***house*** object collecting several numbers, strings, arrays, and other objects to represent a particular instance of a house. The ***house*** variable is assigned an ***object literal*** declared within opening and closing curly braces ***{*** and ***}***. Objects contain pairs of ***properties*** and values separated by commas. Values can be of any datatype including ***Number***, ***String***, ***Boolean***, arrays and other objects. In the example below we declared a ***house*** with 4 ***bedrooms***, 2.5 ***bathrooms*** and 2000 ***squareFeet***. The house has a nested object stored in property ***address*** which contains ***String*** properties such as ***street***, ***city*** and ***state***. The ***owners String*** array declares the names of the owners. To practice with JSON, create a ***House*** component as shown below, import it into the ***Lab3*** component, and confirm it renders as shown in Figure 3.4.9.

app/labs/lab3/House.tsx

```
export default function House() {
  const house = {
    bedrooms: 4,      bathrooms: 2.5,
    squareFeet: 2000,
    address: {
      street: "Via Roma", city: "Roma", state: "RM", zip: "00100",  country: "Italy", },
    owners: ["Alice", "Bob"],
  };
  return (
    <div id="wd-house">
      <h4>House</h4>
      <h5>bedrooms</h5>      {house.bedrooms}
```

```
      <h5>bathrooms</h5>     {house.bathrooms}
      <h5>Data</h5>
      <pre>{JSON.stringify(house, null, 2)}</pre>
      <hr />
    </div>
);}
```

3.4.10 The Spread Operator

The spread operator (...) is used to expand, or copy an iterable object or array into another object or array. In the example below we declare array ***arr1*** and then copy its content (spread) into array ***arr2***. The resulting array ***arr2*** contains the contents of ***arr1***, followed by the rest of the items already declared in ***arr2***. The spread operator can also be applied to objects as illustrated in the following example. Below, ***obj1*** declares an object with three properties ***a***, ***b***, and ***c***. We then spread ***obj1*** onto ***obj2*** so that ***obj2*** ends up with the properties from both ***obj1*** and ***obj2***. When declaring ***obj3***, we first spread ***obj1*** and then declare ***b*** with a value of 4. Since ***obj1*** also has a property called ***b*** with a value of ***2***, there is a collision of properties in ***obj3***. The collision is resolved by keeping the last declaration overriding any previous values, so ***obj3.b*** ends up being ***4***. To practice the spread operator, create the ***Spreader*** component as shown below, import it in the ***Lab3*** component and confirm it renders as shown.

app/labs/lab3/Spreader.tsx

```
export default function Spreading() {
 const arr1 = [ 1, 2, 3 ];
 const arr2 = [ ...arr1, 4, 5, 6 ];
 const obj1 = { a: 1, b: 2, c: 3 };
 const obj2 = { ...obj1, d: 4, e: 5, f: 6 };
 const obj3 = { ...obj1, b: 4 };
 return (
   <div id="wd-spreading">
     <h2>Spread Operator</h2>
     <h3>Array Spread</h3>
     arr1 = { JSON.stringify(arr1) }  <br />
     arr2 = { JSON.stringify(arr2) }  <br />
     <h3>Object Spread</h3>
     { JSON.stringify(obj1) }         <br />
     { JSON.stringify(obj2) }         <br />
     { JSON.stringify(obj3) }         <br />  <hr />
   </div>);}
```

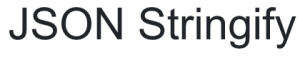


Figure 3.4.8 - JSON Stringify

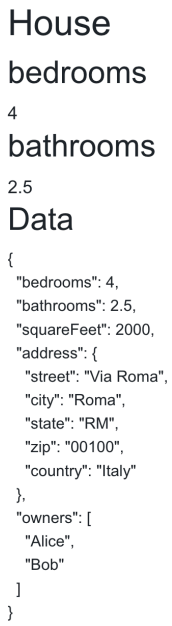


Figure 3.4.9 - JavaScript Object Notation (JSON)

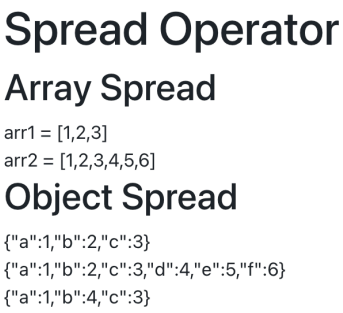


Figure 3.4.10 - The Spread Operator

3.4.11 Destructing

While the spreader operator is used to expand an iterable object into the list of arguments, the destructing operator is used to unpack values from arrays, or properties from objects, into distinct variables. In the example below we declare object ***person*** and array ***numbers***. These can be unpacked, or ***destructed***, into new variables or constants by an object's property name or an array's item position. The curly brackets around constants ***name*** and ***age***, destruct the object ***person*** on the right side of the assignment, and assigns the properties of the same name into the new constants. The constants ***name*** and ***age*** end up having the values of ***person.name*** and ***person.age*** respectively. Essentially it is the equivalent to

```
const name = person.name
const age = person.age
```

While object destructing is based on the names of the properties, destructing arrays is based on the positions of the items. In the example below, we declare the ***numbers*** array and then use the square brackets to destruct the array into new constants ***first***, ***second***, and ***third***. These new constants end up with the values of ***numbers[0]***, ***numbers[1]***, and ***numbers[2]***. Essentially it is equivalent to

```
const first = numbers[0]
const second = numbers[1]
const third = numbers[2]
```

To practice destructing objects and arrays, create component ***Destructing*** as shown below, import it in the ***Lab3*** component, and confirm it renders as shown in Figure 3.4.11 a and b.

app/labs/lab3/Destructing.tsx

```
export default function Destructing() {
 const person = { name: "John", age: 25 };
 const { name, age } = person;
 // const name = person.name
 // const age = person.age
 const numbers = ["one", "two", "three"];
 const [ first, second, third ] = numbers;
 return (
   <div id="wd-destructing">
     <h2>Destructing</h2>
     <h3>Object Destructing</h3>
     const &#123; name, age &#125; = &#123; name: "John", age: 25 &#125;<br /><br />
     name = {name} <br /> age = {age}
     <h3>Array Destructing</h3>
     const [first, second, third] = ["one","two","three"]<br/><br/>
     first = {first}<br />
     second = {second}<br />
     third = {third}<hr />
   </div>
);}
```

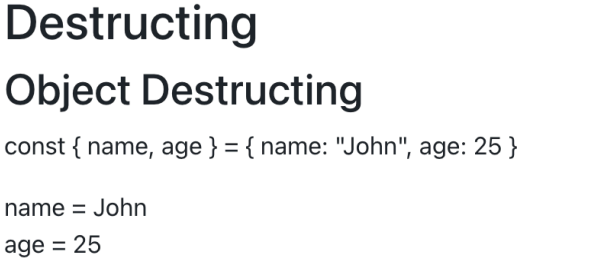


Figure 3.4.11a - Destructing objects

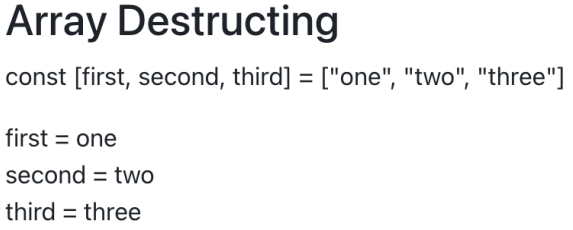


Figure 3.4.11b - Destructing arrays

3.4.12 Destructing Function Parameters

The destructing objects syntax is very popular in React, especially when passing parameters to functions. In the example below we declare two functions ***add*** and ***subtract*** using the new arrow function syntax. The ***add*** function takes two arguments ***a*** and ***b*** and returns the sum of the arguments. The ***subtract*** function takes a single object argument with properties ***a*** and ***b*** with values ***4*** and ***2***. In the argument list declaration, ***subtract*** uses object destructing to declare constants ***a*** and ***b*** which unpacks the values ***4*** and ***2*** from the object argument with properties of the same name. To practice ***function destructing***, copy the code below into a ***FunctionDestructing*** component, import it into the ***Lab3*** component, and confirm it renders as shown in Figure 3.4.12.

app/labs/lab3/FunctionDestructing.tsx

```
export default function FunctionDestructing() {
 const add = (a: number, b: number) => a + b;
 const sum = add(1, 2);
 const subtract = ({ a, b }: { a: number; b: number }) => a - b;
 const difference = subtract({ a: 4, b: 2 });
 return (
   <div id="wd-function-destructing">
     <h2>Function Destructing</h2>
     const add = (a, b) =&gt; a + b;<br />
     const sum = add(1, 2);<br />
     const subtract = (&#123; a, b &#125;) =&gt; a - b;<br />
     const difference = subtract(&#123; a: 4, b: 2 &#125;);<br/>
     sum = {sum}<br />
     difference = {difference} <hr />
   </div>
);}
```

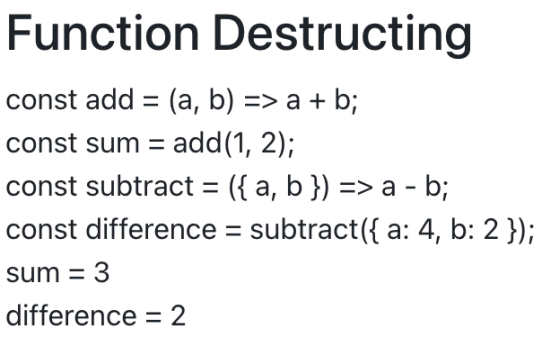


Figure 3.4.12 - Destructing Function Parameters

3.4.13 Destructing Imports

Let's create a simple library to illustrate various ways of importing the functions and constants declared in the ***Math*** library below. The functions ***add***, ***subtract***, ***multiply***, and ***divide*** are all exported with the ***export*** keyword so that they can be imported individually. The ***Math*** constant declares an object containing references to the local functions. We export the ***Math*** object as the ***default export*** so that the functions can be imported as a single object map.

app/labs/lab3/Math.ts

```
export function add(a: number, b: number): number      { return a + b; }
export function subtract(a: number, b: number): number { return a - b; }
export function multiply(a: number, b: number): number { return a * b; }
export function divide(a: number, b: number): number   { return a / b; }
const Math = {
  add,
  subtract,
```

```
  multiply,
  divide,
};
export default Math;
```

To demonstrate how the functions can be imported in several ways, create the ***DestructingImports*** component below. The component implements a table we're going to fill out with three different ways we can import the functions from the ***Math*** library we implemented above. First, the functions can be imported as the single ***Math*** object that contains references to all the functions as ***import Math from "./Math"***. Then we can access each of the functions through the ***Math*** instance as ***Math.add()***, ***Math.subtract()***, etc. An alternative way to import the functions as a single object is using the ***import * as NAME from "./Math"***, where ***NAME*** is a custom local object name, e.g., ***Matematica***. We can then using the object's name to invoke the functions as ***Matematica.add()***, ***Matematica.subtract()***, etc. Finally, the functions can be imported individually by destructing the exported functions as ***import {add, subtract, multiply, divide} from "./Math"***.

app/labs/lab3/DestructingImports.tsx

```
import Math, { add, subtract, multiply, divide } from "./Math";
import * as Matematica from "./Math";
export default function DestructingImports() {
  return (
    <div id="wd-destructuring-imports">
      <h2>Destructing Imports</h2>
      <table className="table table-sm">
        <thead>
          <tr>
            <th>Math</th>
            <th>Matematica</th>
            <th>Functions</th>
          </tr>
        </thead>
        <tbody>
          {/* see next code block */}
        </tbody>
      </table>
      <hr />
    </div>
);}
```

Here are several examples demonstrating using the functions through the objects ***Math***, ***Matematica***, and then using the functions individually. Implement the ***Math.ts*** library and the ***DestructingImports*** component and confirm it renders as shown. Complete missing code.

app/labs/lab3/DestructingImports.tsx

```
<tr>
  <td>Math.add(2, 3) = {Math.add(2, 3)}</td>
  <td>Matematica.add(2, 3) =
     {Matematica.add(2, 3)}</td>
  <td>add(2, 3) = {add(2, 3)}</td>
</tr>
<tr>
  <td>Math.subtract(5, 1) = {Math.subtract(5, 1)}</td>
  <td>Matematica.subtract(5, 1) =
     {Matematica.subtract(5, 1)}</td>
  <td>subtract(5, 1) = {subtract(5, 1)}</td>
</tr>
```

Destructing Imports

Math	Matematica	Functions
Math.add(2, 3) = 5	Matematica.add(2, 3) = 5	add(2, 3) = 5
Math.subtract(5, 1) = 4	Matematica.subtract(5, 1) = 4	subtract(5, 1) = 4
Math.multiply(3, 4) = 12	Matematica.multiply(3, 4) = 12	multiply(3, 4) = 12
Math.divide(8, 2) = 4	Matematica.divide(8, 2) = 4	divide(8, 2) = 4

Figure 3.4.13 - Destructing Imports

3.5 Dynamic Styling

React can generate content dynamically based on algorithms written in JavaScript. We can also dynamically style the content by programmatically controlling the classes and styles applied to the content. In the next couple of exercises we first learn to work with classes and then with styles.

3.5.1 Working with HTML classes

Let's start practicing simple things, like classes and styles. Under the ***labs/lab3*** folder, create a new component ***Classes*** with a matching styling file. From the ***Lab3*** component, import the new ***Classes*** component and confirm the component renders as shown in Figure 3.5.1a.

app/labs/lab3/Classes.tsx

```
import './Classes.css';
export default function Classes() {
  return (
    <div>
      <h2>Classes</h2>
      <div className="wd-bg-yellow
            wd-fg-black wd-padding-10px">
        Yellow background  </div>
      <div className="wd-bg-blue
            wd-fg-black wd-padding-10px">
        Blue background    </div>
      <div className="wd-bg-red
            wd-fg-black wd-padding-10px">
        Red background     </div><hr/>
    </div>
  )
};
```

app/labs/lab3/Classes.css

```
.wd-bg-yellow    { background-color: lightyellow; }
.wd-bg-blue      { background-color: lightblue;   }
.wd-bg-red       { background-color: lightcoral;  }
.wd-bg-green     { background-color: lightgreen;  }
.wd-fg-black     { color: black;                  }
.wd-padding-10px { padding: 10px;                 }
```

The previous example used static classes such as ***wd-bg-yellow***. Instead we could calculate the class we want to apply based on some logic. Here's an example of creating the classes dynamically by concatenating a ***color*** constant. Refresh the screen and confirm components render as shown in Figure 3.5.1b.

app/labs/lab3/Classes.tsx

```
export default function Classes() {
  const color = 'blue';
  return (
    <div id="wd-classes">
      <h2>Classes</h2>
      <div className={`wd-bg-${color} wd-fg-black wd-padding-10px`}>
        Dynamic Blue background
      </div> ...
  );
}
```

Even more interesting is using expressions to conditionally choose between a set of classes. The example below uses either a ***red*** or ***green*** background based on the ***dangerous*** constant. Try with ***dangerous true*** and ***false*** and confirm it renders red or green as expected.

app/labs/lab3/Classes.tsx

```
export default function Classes() {
 const color = 'blue';
 const dangerous = true;
 return (
   <div id="wd-classes">
     <h2>Classes</h2>
     <div className={`${dangerous ? 'wd-bg-red' : 'wd-bg-green'} wd-fg-black wd-padding-10px`}>
       Dangerous background
     </div> ...
 );
}
```

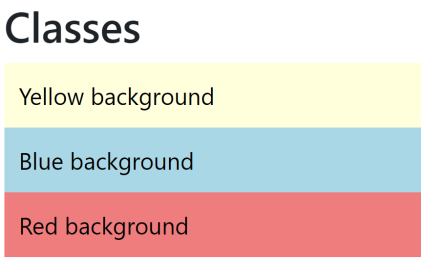


Figure 3.5.1a - Working with HTML classes

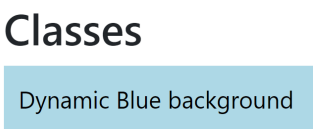


Figure 3.5.1b - Working with HTML classes

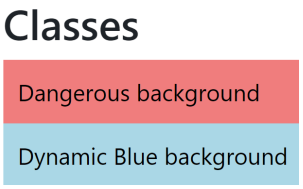


Figure 3.5.1c - Working with HTML classes

3.5.2 Working with the HTML Style attribute

In React, the ***styles*** attribute accepts a ***JSON*** object where the properties are ***CSS*** properties and the values are CSS values. To practice how this works, implement the ***Styles*** component below and then import it into the ***Lab3*** component. The ***Styles*** component declares constant JSON objects that can be applied to elements using the ***styles*** attribute. Alternatively, the styles attribute accepts a JSON literal object instance which results in a weird syntax of double curly brackets highlighted below. Refresh the browser and confirm the browser renders as expected.

labs/lab3/Styles.tsx

```
export default function Styles() {
  const colorBlack = { color: "black" };
  const padding10px = { padding: "10px" };
  const bgBlue = { "backgroundColor": "lightblue",
                   "color": "black",   ...padding10px
  };
  const bgRed = {
    "backgroundColor": "lightcoral",
    ...colorBlack,   ...padding10px
  };
  return(
    <div id="wd-styles">
      <h2>Styles</h2>
      <div style={{backgroundColor: "lightyellow", color: "black", padding: "10px" }}>
        Yellow background</div>
```

```
      <div style={ bgRed }> Red background </div>
      <div style={ bgBlue }>Blue background</div>
    </div>
);};
```

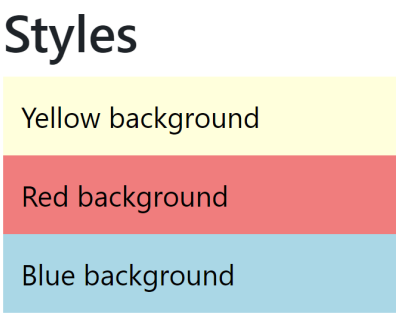


Figure 3.5.2 - Working with the HTML Style attribute

3.6 Client and Server Components

In Next.js, all components are ***Server Components*** by default. This means they execute only on the server during rendering, producing HTML that is sent to the browser. ***Server Components*** are fast, secure, and can directly access server-only resources (like the filesystem or environment variables), but they cannot use browser-specific features like interactivity, state, or DOM APIs. If you need interactivity or browser APIs, you must explicitly turn a component into a ***Client Component*** by adding ***'use client'*** at the top of the file. ***Client Components*** run only in the browser, allowing hooks, event handlers, and browser globals — but they lose direct server access. The two simple examples below highlight exactly what each side can (and cannot) do.

3.6.1 Client Components

This ***Client Component*** below is marked with the ***use client*** directive at the top, which forces it to execute exclusively in the browser rather than on the server. This allows safe use of browser-only features, such as hooks from next/navigation.

In this example, the component uses the ***usePathname()*** hook to read the current route. This hook is client-only and would cause a server-side error if the component were treated as a Server Component (i.e., without "use client"). Removing the directive would result in a build or runtime failure because ***usePathname()*** is not available during server rendering.

The code renders a simple heading and displays the current pathname, clearly demonstrating that the component is running on the client and has access to browser-specific Next.js APIs.

app/labs/lab3/ClientComponentDemo.tsx

```
"use client";
import { usePathname } from "next/navigation";

export default function ClientComponentDemo() {

 // This runs ONLY in the browser
 const pathname = usePathname();

 return (
   <div>
     <h1>Client Component Demo</h1>
     <p>Current pathname: {pathname}</p>
```

```
    </div>
  );
}
```

3.6.2 Server Components

By default, Next.js pages and components are ***Server Components*** and are marked by omitting the ***use client*** directive. The server component below omits the ***use client*** directive, making it execute exclusively on the server by default in Next.js. It demonstrates server-only capabilities by accessing Node.js globals like the process object for system details and using ***fs.readdirSync()*** to synchronously list files from the project's root directory on the server's filesystem. Adding ***use client*** would cause a build failure, as APIs like process and fs are unavailable in the browser environment.

app/labs/lab3/ServerComponentDemo.tsx

```
// No 'use client' → this is a Server Component (default)
import fs from "node:fs";
export default function ServerComponentDemo() {

  // Server components can access server-only APIs like 'process' and 'fs'
  const processInfo = {
    platform: process.platform,
    nodeVersion: process.version,
    memoryUsage: process.memoryUsage(),
    cwd: process.cwd(),
  };

  // Get the current time on the server
  const serverRenderTime = new Date().toLocaleTimeString();

  // List files in the current working directory (project root)
  const projectRoot = process.cwd();
  let files: string[] = [];
  try {
    files = fs.readdirSync(projectRoot);
  } catch (error) {
    console.error("Error reading project directory:", error);
    files = [];
  }
  return (
    <div>
      <h1>Server Component Demo</h1>
      <h2>Server Render Time</h2>
      <p>Rendered on server at: {serverRenderTime}</p>
      <h2>Server Information</h2>
      <pre>{JSON.stringify(processInfo, null, 2)}</pre>
      <h2>Filesystem Access Demo</h2>
      <pre>{JSON.stringify(files, null, 2)}</pre>
    </div>
  );
}
```

Include the components above in the lab page and confirm they render as shown below.

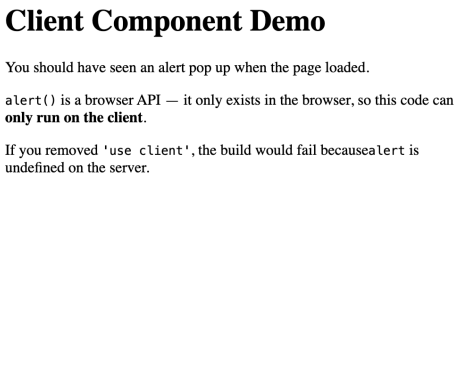


Figure 3.6a - Client Component

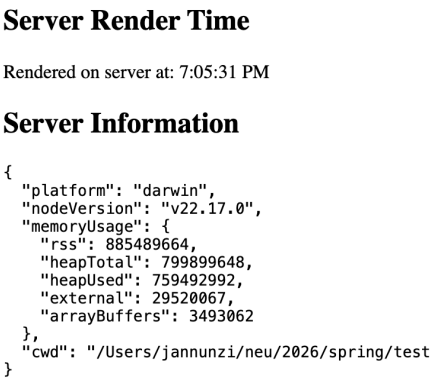


Figure 3.6b - Server Component

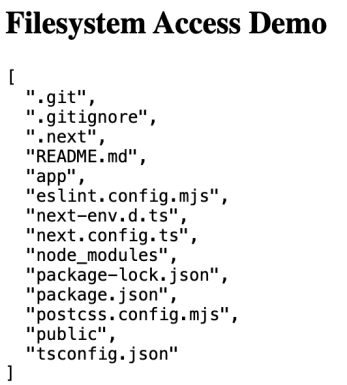


Figure 3.6c - File Access

3.7 Parameterizing Components

React components can be parameterized by using the familiar HTML attribute syntax, which passes attribute values to the component's function as an object map parameter. The following ***Add*** component can receive properties ***a*** and ***b*** deconstructed from the attributes of its equivalent HTML attributes syntax. Implement the ***Add*** component below and confirm that passing it ***a=3*** and ***b=4*** results in ***a + b = 7***. Note that the values of ***a*** and ***b*** are destructed from the object parameter in the ***Add*** function parameter list.

app/labs/lab3/Add.tsx

```
export default function Add(
    { a, b }: { a: number; b: number }) {
  return (
    <div id="wd-add">
      <h4>Add</h4>a = {a}
      b = {b}          <br />
      a + b = {a + b} <hr />
    </div>
  );
}
```

app/labs/lab3/page.tsx

```
import Add from "./Add";
export default function Lab3() {
  return (
    <div id="wd-lab3" className="container">
      <h3>Lab 3</h3>
      ...
      <Add a={3} b={4} />
    </div>
  );
}
```

3.7.1 Child Components

In the previous section we discussed passing data to a component through attributes. Another way to pass data to a component is in its body, that is, between the opening and closing tag of the element. In HTML it's common to wrap content with specific tags to add certain formatting. For instance the tags ***h1*** and ***p*** shown below format the content in their bodies with specific font sizes and margins. Basically these elements take the content in the body and return a transformed version of the content.

```
<h1>This content is formatted as a heading of size 1</h1>
<p>This content is formatted as a paragraph</p>
```

We can implement ***React*** components to take content in their body and return a new version of that content. The content in the body of a ***React*** component is passed to the component function as parameter called ***children***. For instance, the component below takes a number in its body and returns the square of the number. Import the new component, use to compute the square of 4 and confirm it renders the correct result

app/labs/lab3/Square.tsx ***app/labs/lab3/page.tsx***

```
import React, { ReactNode } from "react";
export default function Square({ children }: { children: ReactNode }) {
  const num = Number(children);
  return <span id="wd-square">{num * num}</span>;
}
```

```
import Square from "./Square";
export default function Lab3() {
  return (
    <div id="wd-lab3">
      <h3>JavaScript</h3>
      ...
      <h4>Square of 4</h4>
      <Square>4</Square>
      <hr />
    </div>
  );
}
```

Here's another example that highlights the content in its body making the content red on yellow. The example below receives the ***children*** property as a property and then renders it in the return statement. The ***children*** property is a special property that contains the ***body*** of component when using its opening and closing tags as shown below.

app/labs/lab3/Highlight.tsx

```
import { ReactNode } from "react";
export default function Highlight({ children }: { children: ReactNode }) {
  return (
    <span id="wd-highlight" style={{ backgroundColor: "yellow", color: "red" }}>
      {children}
    </span>
  );
}
```

Implement the ***Highlight*** component as shown above, import it in ***Lab3***, and confirm it highlights the content shown.

app/labs/lab3/page.tsx

```
...
import Highlight from "./Highlight";

export default function Lab3() {
 return (
   <div id="wd-lab3">
     <h3>Lab 3</h3>
     ...
     <Highlight>
       Lorem ipsum dolor sit amet consectetur adipisicing elit. Suscipitratione eaque illo minus cum, saepe
totam
       vel nihil repellat nemo explicabo excepturi consectetur. Modi omnis minus sequi maiores, provident
voluptates.
     </Highlight>
   </div>
 );
}
```

3.7.2 Working with the Pathname

The ***usePathname*** hook from ***Next.js*** returns the current URL's ***pathname***, enabling dynamic UI logic such as highlighting navigation elements or conditionally rendering content. In the example below, the pathname is used to check if the URL ends with ***/lab1***, ***/lab2***, or ***/lab3***, applying an ***active*** class to highlight the corresponding navigation pill. Verify that clicking each pill highlights the correct tab in the UI. ***Note*** the ***"use client"*** directive at the top of the file, which ensures this code runs on the client side, as ***usePathname*** relies on browser APIs to access the URL from the browser's ***address bar***.

app/Labs/TOC.tsx

```
"use client";
import { Nav, NavItem, NavLink } from "react-bootstrap";
import Link from "next/link";
import { usePathname } from "next/navigation";
export default function TOC() {
 const pathname = usePathname();
 return (
   <Nav variant="pills">
     <NavItem>
       <NavLink href="/labs" as={Link} className={`nav-link ${pathname.endsWith("labs") ? "active" : ""}`}>
         Labs </NavLink> </NavItem>
     <NavItem>
       <NavLink href="/labs/lab1" as={Link} className={`nav-link ${pathname.endsWith("lab1") ? "active" : ""}`}>
         Lab 1 </NavLink> </NavItem>
     <NavItem>
       <NavLink href="/labs/lab2" as={Link} className={`nav-link ${pathname.endsWith("lab2") ? "active" : ""}`}>
         Lab 2 </NavLink> </NavItem>
     <NavItem>
       <NavLink href="/labs/lab3" as={Link} className={`nav-link ${pathname.endsWith("lab3") ? "active" : ""}`}>
         Lab 3 </NavLink> </NavItem>
     <NavItem>
       <NavLink href="/" as={Link}>
         Kambaz </NavLink> </NavItem>
     <NavItem>
       <NavLink href="https://github.com/jannunzi">My GitHub</NavLink></NavItem>
   </Nav>
 );}
```

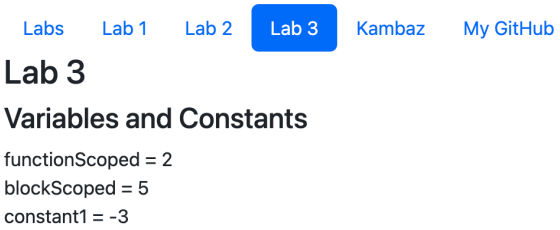


Figure 3.7.2 - Working with Pathname

3.7.3 Encoding Path Parameters

The URL can also be used to encode parameters when navigating between screens. Page components can parse parameters from the path using the ***useParams*** Next.js hook. The ***AddPathParameters*** page component below parses parameters ***a*** and ***b*** from the path and calculates the arithmetic addition of the parameters.

app/Labs/Lab3/add/[a]/[b]/page.tsx

```
"use client"
import { useParams } from "next/navigation";
export default function AddPathParameters() {
  const { a, b } = useParams();
  return (
    <div id="wd-add"> <h4>Add Path Parameters</h4>
      {a} + {b} = {parseInt(a as string) + parseInt(b as string)}
    </div>
);}
```

The ***PathParameters*** component below declares two links that encode parameters ***a*** and ***b***. The first link encodes values 1 and 2 for parameters ***a*** and ***b***, whereas the second link encodes values 3 and 4 for parameters ***a*** and ***b***. Import ***PathParameters*** component in your ***Lab3*** component and confirm clicking the first link, the URL matches the ***path*** of the ***AddPathParameters*** page component and renders the component with parameters ***a=1*** and ***b=2*** and so it renders ***1 + 2 = 3***. Confirm clicking the second link sets ***a=3*** and ***b=4*** and so it renders ***3 + 4 = 7***.

app/Labs/Lab3/PathParameters.tsx

```
import Link from "next/link";
export default function PathParameters() {
 return (
   <div id="wd-path-parameters">
     <h2>Path Parameters</h2>
     <Link href="/labs/lab3/add/1/2">1 + 2</Link> <br />
     <Link href="/labs/lab3/add/3/4">3 + 4</Link>
   </div>
);}
```

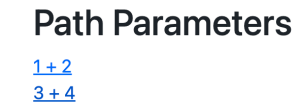


Figure 3.6.3a - Encoding Path Parameters

Figure 3.6.3b - Encoding Path Parameters

3.8 Rendering a Data Structure

Let's bring together several of the concepts covered so far and implement a ***Todo*** list application that renders a list of todos dynamically using React. In a new directory ***app/labs/lab3/todos***, implement the ***TodoItem*** component in a ***TodoItem.tsx*** file as shown below. Import the component into the ***Lab3*** component and confirm that it renders as shown in Figure 3.8 a and b.

app/Labs/Lab3/todos/TodoItem.tsx

```
const TodoItem = ( { todo = { done: true, title: 'Buy milk', status: 'COMPLETED' } }) => {
 return (
   <ListGroupItem>
     <input type="checkbox" className="me-2"
            defaultChecked={todo.done}/>
     {todo.title} ({todo.status})
   </ListGroupItem>
);}
export default TodoItem;
```

Create a JSON file ***todos.json*** that contains an array of todos as shown below.

app/Labs/Lab3/todos/todos.json

```
[
  { "title": "Buy milk",          "status": "CANCELED",     "done": true   },
  { "title": "Pickup the kids",   "status": "IN PROGRESS",  "done": false  },
  { "title": "Walk the dog",      "status": "DEFERRED",     "done": false  }
]
```

Now let's implement a ***TodoList*** component that renders the array of todos as shown below. Import the component in ***labs/lab3/page.tsx***, refresh the browser, and confirm the ***TodoList*** renders a list of checkboxes and todo items.

app/Labs/Lab3/todos/TodoList.tsx *Browser*

```
import TodoItem from "./TodoItem";
import todos from "./todos.json";
export default function TodoList() {
 return(
   <>
     <h3>Todo List</h3>
     <ListGroup>
```

```
      { todos.map(todo => {
          return(<TodoItem todo={todo}/>);   })}
    </ListGroup><hr/>
  </>
);}
```

Figure 3.8a - Rendering a Data Structure

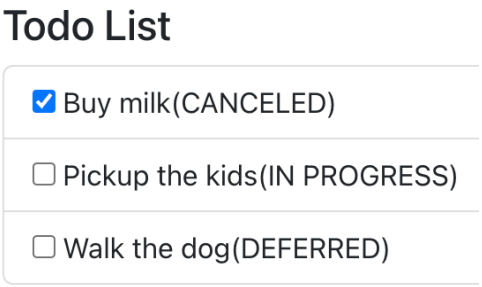


Figure 3.8b - Rendering a Data Structure

3.9 Debugging

The ***Web Dev Tools*** provide several tools to analyze various types of source code common in the Web development process. So far we've looked at the ***Elements*** and ***Styles*** tab where we can analyze the ***DOM*** data structure generated by the browser when it parsed the ***HTML*** or content dynamically generated by ***JavaScript***. Now that we have been implementing functions and algorithms, we should become familiar with the ***Console*** and ***Source*** tabs.

3.9.1 Writing to the Console from JavaScript

A useful feature of modern browsers is providing a development environment where developers can analyze the performance of their scripts. One way to analyze scripts are behaving correctly is to write output to the ***console*** from within scripts. To practice writing to the ***console***, bring up the console on the browser by right clicking on the page and selecting ***Inspect***. The page will split in half displaying useful developer tools similar as shown below.

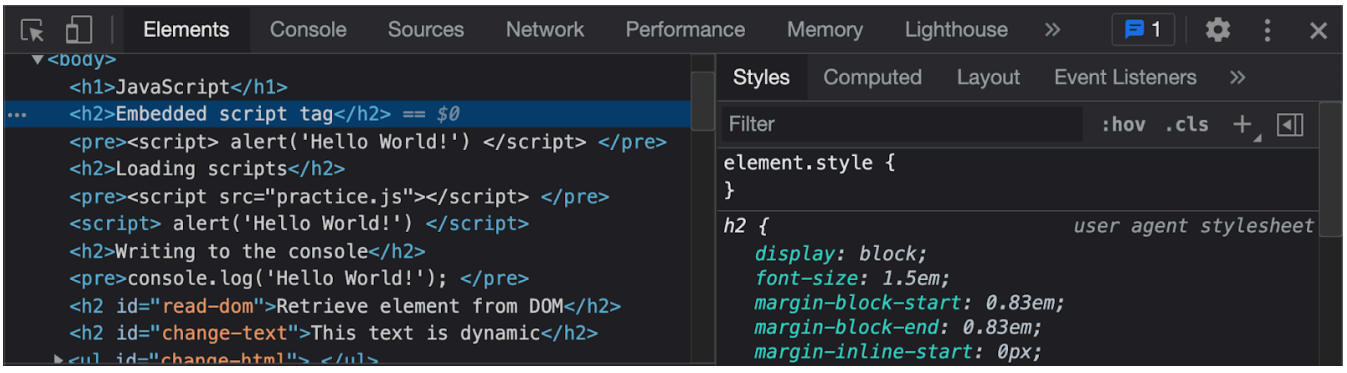


Figure 3.9.1a - Writing to the Console from JavaScript

Click on the ***Console*** tab where we will be logging to throughout this chapter.

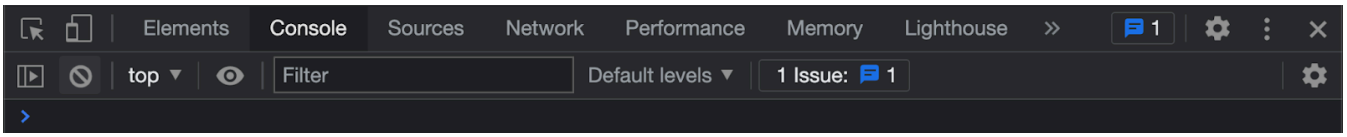


Figure 3.9.1b - Writing to the Console from JavaScript

Add a ***console.log()*** statement to the ***Lab3*** component, reload the screen and confirm that "***Hello World!***" is displayed in the console.

app/labs/lab3/page.tsx

```
export default function Lab3() {
  console.log('Hello World!');
  return(
    <div>
      <h3>JavaScript</h3>
         ...
    </div>
);}
```

Console

```
Hello World!
```

We can also display ***JSON*** data to confirm it is what we expect. Add a ***console.log()*** statement to print the ***house*** JSON object in the ***House*** component as shown below. Confirm the ***house*** object prints to the console.

app/labs/lab3/House.tsx

```
export default function House() {
  const house = {
    ...
  };
  console.log(house);
  return (
    <div id="wd-house">
      ...
    </div>
);}
```

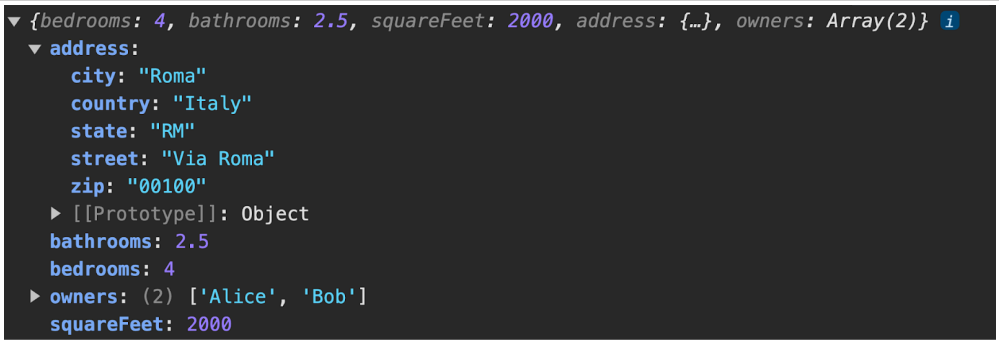


Figure 3.9.1c - Writing to the Console from JavaScript

3.10 Implementing a Data Driven Kambaz Application

Previous chapters implemented the ***Kambaz*** application using ***React*** components and screens that aggregated their output into a single application. The ***Kambaz*** application is currently displaying static content that can not be modified and does not depend on the changing context. For instance, the ***Dashboard*** screen should be different based on who logs in, and ***Courses*** screen should display different ***Modules*** and ***Assignments***, depending on which course a user selects in the ***Dashboard*** screen. This chapter revisits the implementation by making it ***data driven***, that is the content will be dynamic based on ***JSON*** data files that determine the courses displayed in the ***Dashboard***, and the ***Modules*** and ***Assignments*** displayed as users select a particular course. Before beginning, confirm the ***Kambaz*** landing screen implementation is similar to the code shown below. In particular, make sure the ***account/signin*** is the landing page.

app/(kambaz)/page.tsx

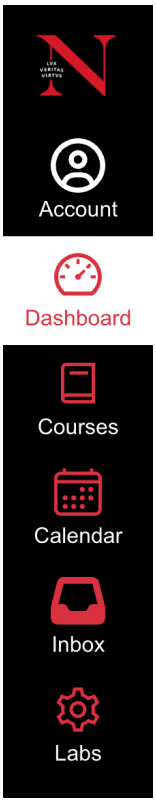
```
import { redirect } from "next/navigation";
export default function Kambaz() {
 redirect("/account/signin");
}
```

3.10.1 Data Driven Kambaz Navigation

The current implementation of the ***KambazNavigation*** component consists of a hardcoded list group of links to navigate to the ***Dashboard*** and other places in ***Kambaz***. Instead of hardcoding the links, create a data structure that configures the ***label***s, ***path***s, and ***icon***s as an array and then map over the data structure creating the links dynamically. Here's an example of how to implement the ***Kambaz Navigation*** component. Confirm the sidebar renders as shown below and the navigation still works. Note that the ***Courses*** link has been intentionally configured to navigate to the ***Dashboard***, since it only makes sense to navigate to the ***Courses*** screen from the ***Dashboard***.

app/(kambaz)/Navigation.tsx

```
"use client"
import { AiOutlineDashboard } from "react-icons/ai";
import { IoCalendarOutline } from "react-icons/io5";
import { LiaBookSolid, LiaCogSolid } from "react-icons/lia";
import { FaInbox, FaRegCircleUser } from "react-icons/fa6";
import { usePathname } from "next/navigation";
import Link from "next/link";
export default function KambazNavigation() {
  const pathname = usePathname();
  const links = [
    { label: "Dashboard", path: "/dashboard", icon: AiOutlineDashboard },
    { label: "Courses",   path: "/dashboard", icon: LiaBookSolid },
    { label: "Calendar",  path: "/Calendar",  icon: IoCalendarOutline },
    { label: "Inbox",     path: "/Inbox",     icon: FaInbox },
    { label: "Labs",      path: "/labs",              icon: LiaCogSolid },
  ];
  return (
    <ListGroup id="wd-kambaz-navigation" style={{width: 120}}
         className="rounded-0 position-fixed bottom-0 top-0 d-none d-md-block bg-black z-2">
      <ListGroupItem id="wd-neu-link" target="_blank" href="https://www.northeastern.edu/"
        action className="bg-black border-0 text-center">
        <img src="/images/NEU.png" width="75px" /></ListGroupItem>
      <ListGroupItem as={Link} href="/account"
        className={`text-center border-0 bg-black
            ${pathname.includes("Account") ? "bg-white text-danger" : "bg-black text-white"}`}>
        <FaRegCircleUser
          className={`fs-1 ${pathname.includes("Account") ? "text-danger" : "text-white"}`} />
        <br />
        Account
      </ListGroupItem>
      {links.map((link) => (
        <ListGroupItem key={link.path} as={Link} href={link.path}
          className={`bg-black text-center border-0
            ${pathname.includes(link.label) ? "text-danger bg-white":"text-white bg-black"}`}>
          {link.icon({ className: "fs-1 text-danger"})}
          <br />
          {link.label}
        </ListGroupItem>
      ))}
    </ListGroup>
);}
```



3.10.2 Implementing a Kambaz "Database"

Implement a data file that will contain all the data needed in the ***Kambaz*** application. Start by adding courses in a ***courses.json*** file under a new ***Database*** folder. Download a sample data file containing the courses from my gist on GitHub. Save the file to ***app/(kambaz)/database/courses.json***. Create a database file that contains all the data needed for the ***Kambaz*** application. For now we just have the courses, but later sections will add users, assignments, modules, etc.

app/(kambaz)/database/index.ts

```
import courses from "./courses.json";
export {  courses  };
```

3.10.3 Data Driven Dashboard Screen

Based on your implementation of the ***Dashboard*** in previous chapters, refactor the component so that it dynamically renders the ***courses*** in the ***Database***. The following code is an example of how you can implement the ***Dashboard*** component dynamically mapping through an array of ***courses***. Confirm that the ***Dashboard*** component renders the courses as a grid and that clicking on a course navigates to the ***Home*** screen, but now the ***URL*** encodes the ***ID*** of the course.

app/(kambaz)/dashboard/page.tsx

```
import Link from "next/link";
import * as db from "../database";
export default function Dashboard() {
 const courses = db.courses;
 return (
  <div id="wd-dashboard">
   <h1 id="wd-dashboard-title">Dashboard</h1> <hr />
   <h2 id="wd-dashboard-published">Published Courses ({courses.length})</h2> <hr />
   <div id="wd-dashboard-courses">
    <Row xs={1} md={5} className="g-4">
     {courses.map((course) => (
     <Col className="wd-dashboard-course" style={{ width: "300px" }}>
      <Card>
       <Link href={`/courses/${course._id}/home`}
        className="wd-dashboard-course-link text-decoration-none text-dark" >
        <CardImg src="/images/reactjs.jpg" variant="top" width="100%" height={160} />
        <CardBody className="card-body">
         <CardTitle className="wd-dashboard-course-title text-nowrap overflow-hidden">
          {course.name} </CardTitle>
         <CardText className="wd-dashboard-course-description overflow-hidden" style={{height:"100px"}}>
          {course.description} </CardText>
         <Button variant="primary"> Go </Button>
        </CardBody>
       </Link>
      </Card>
     </Col>
    ))}
   </Row>
  </div>
 </div>);}
```

The ***Link*** component is used to create the hyperlinks and encode the course's ID as part of the path. This can then be used in other components to parse the ID from the path and display content specific to the selected course. Confirm that clicking on different courses encodes the corresponding course ID in the path.

3.10.4 Data Driven Courses Screen

Now that clicking on a course on the ***Dashboard*** encodes the course's ***ID*** in the URL path, the ***Courses*** layout can be refactored to retrieve the course's ***ID*** from the dynamic route parameter and render the corresponding course's name. In Next.js, the Courses layout (located at ***app/(kambaz)/courses/[cid]/layout.tsx***) accesses the course ID (***cid***) through the ***params*** object, which is provided by Next.js for dynamic routes. Using the cid parameter, the course can be looked up from the ***Database*** by matching the ***_id*** field. Refactor the Courses layout to render the name of the course clicked on in the Dashboard, as shown below.

app/(kambaz)/courses/[cid]/layout.tsx

```
import { FaAlignJustify } from "react-icons/fa6";
import { courses } from "../../database";

export default function CoursesLayout({ children, params }: ...) {
  const { cid } = await params;
  const course = courses.find((course) => course._id === cid);
  return (
    <div id="wd-courses">
      <h2 className="text-danger">
        <FaAlignJustify className="me-4 fs-4 mb-1" />
        {course?.name}
      </h2>
      ...
    </div>
);}
```

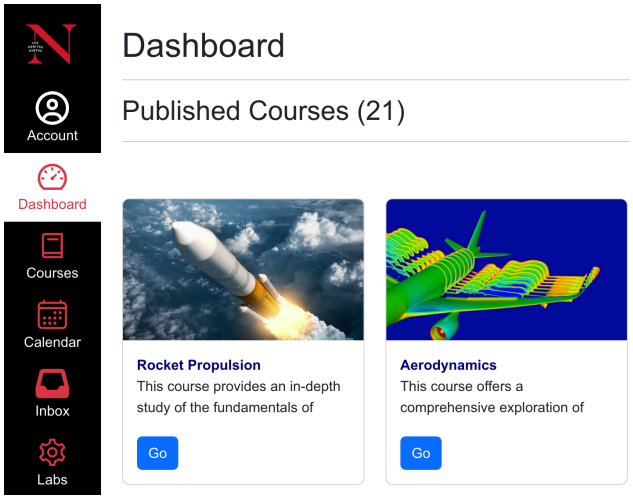


Figure 3.10.3 - Data Driven Dashboard Screen

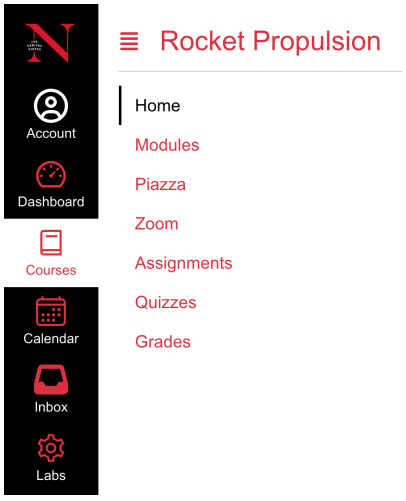


Figure 3.10.4 - Data Driven Courses Screen

3.10.5 Data Driven Course Navigation (On Your Own)

Based on the earlier ***Data Driven Kambaz Navigation*** example, refactor the ***Course Navigation*** sidebar to use the following array instead of hardcoded links:

```
const links = ["Home", "Modules", "Piazza", "Zoom", "Assignments", "Quizzes", "Grades", "People"];
```

In Next.js, retrieve the current course's ID (***cid***) from the dynamic route parameter using the ***params*** object in the layout or page component. Use the ***usePathname*** hook from next/navigation to get the current pathname and determine which link is active. Ensure the links highlight when clicked, and the URL encodes the course's ID in the path (e.g., ***/courses/[cid]/home***). Navigation to the ***Home***, ***Modules***, ***Assignments***, and ***People*** screens should function as before, using Next.js's file-based routing. Verify the sidebar highlights the correct link based on the current route.

3.10.6 Implementing the Breadcrumb Component

A ***breadcrumb*** is a graphical representation that helps users know where they are within a set of screens. For instance the course name at the top of the ***Courses*** screen lets users know they are in a particular course. As they click through the links in the ***Courses Navigation*** sidebar, we can append the name of the section to the name of the course to further indicate what section are we in. The screenshots below illustrate navigating to the ***Home***, ***Modules***, and ***Assignments*** screens. In the ***Courses*** component, implement the breadcrumbs as shown below.

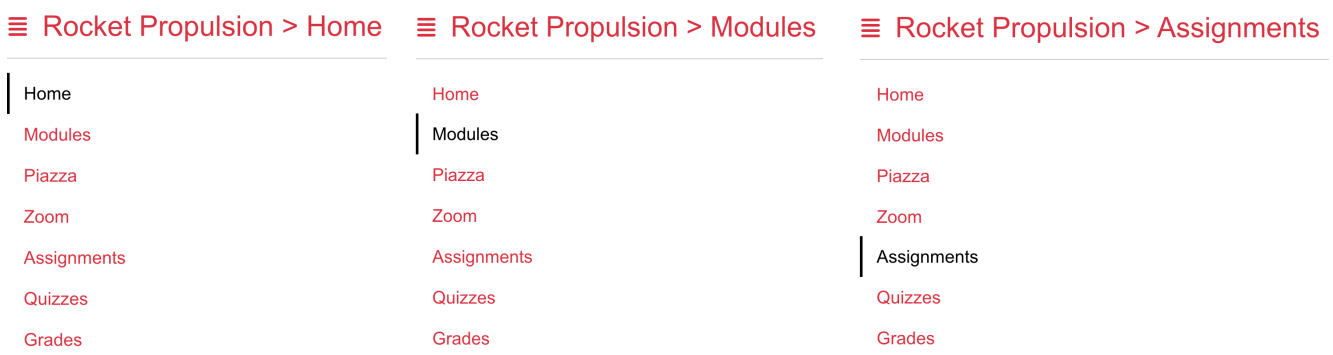


Figure 3.8.6 - Implementing the Breadcrumb Component

Below is a suggestion on how you could implement the breadcrumb.

app/(kambaz)/courses/[cid]/Breadcrumb.tsx

```
"use client";
import React from "react";
import { usePathname } from "next/navigation";
export default function Breadcrumb({ course }: { course: { name: string } | undefined; }) {
 const pathname = usePathname();
 return (
   <span>
     Course {course?.name} &gt; {pathname.split("/").pop()}
   </span>
);}
```

3.10.7 Data Driven Modules Screen

Currently the ***Modules*** and ***Home*** screen render the same course modules regardless which course you click on in the ***Dashboard***. In this section we're going to modify the ***Modules*** screen so that we display the corresponding modules for the selected course. Each course has a different set of modules and each module has its set of lessons. Use the data provided and save it to ***app/(kambaz)/database/modules.json*** file containing at least 3 modules for each course. Include the modules in the ***Database*** as shown below.

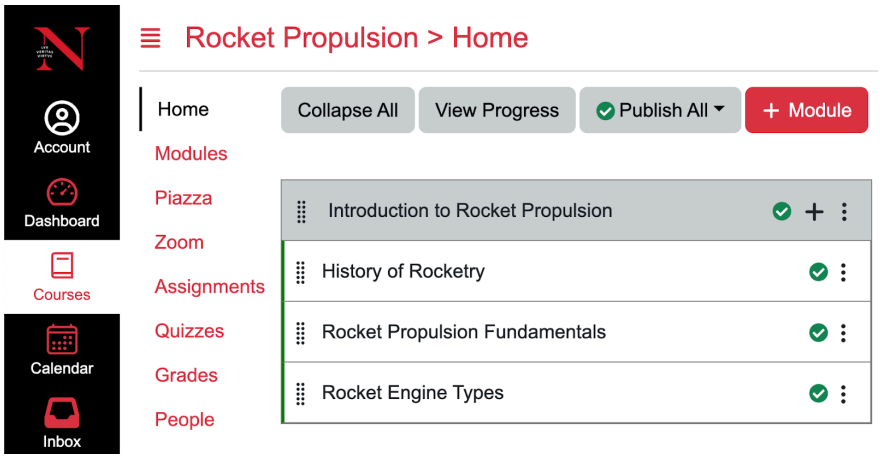


Figure 3.10.7 - Data Driven Modules Screen

app/(kambaz)/database/index.ts

```
import courses from "./courses.json";
import modules from "./modules.json";
export { courses, modules };
```

Below is an example of how you can use the ***modules*** in the ***Database*** to render a list of modules. Use ***useParams()*** to retrieve the course ID from the URL and then retrieve the corresponding modules for the course. Confirm that navigating to different courses, the corresponding modules and lessons are rendered as shown in Figure 3.10.7.

app/(kambaz)/courses/[cid]/modules/page.tsx

```
"use client"
import { useParams } from "next/navigation";
import * as db from "../../database";
export default function Modules() {
  const { cid } = useParams();
  const modules = db.modules;
  return (
      ...
      <ListGroup id="wd-modules" className="rounded-0">
        {modules
          .filter((module: any) => module.course === cid)
          .map((module: any) => (
          <ListGroupItem className="wd-module p-0 mb-5 fs-5 border-gray">
            <div className="wd-title p-3 ps-2 bg-secondary">
              <BsGripVertical className="me-2 fs-3" /> {module.name} <ModuleControlButtons /> </div>
            {module.lessons && (
              <ListGroup className="wd-lessons rounded-0">
                {module.lessons.map((lesson: any) => (
                  <ListGroupItem className="wd-lesson p-3 ps-1">
                    <BsGripVertical className="me-2 fs-3" /> {lesson.name} <LessonControlButtons />
</ListGroupItem>
                ))}</ListGroup>)}</ListGroupItem>))}</ListGroup>...);}
```

3.10.8 Data Driven Assignments Screen (On Your Own)

Based on your implementation of the ***Assignments*** screen in previous chapters, refactor the component so that it renders the assignments for the currently selected course. Use the data provided as an example to create an

assignments.json file which contains various assignments for the courses in the ***courses.json*** file provided. Include the ***assignments*** in the ***Database*** as shown below.

app/(kambaz)/database/index.ts

```
import courses from "./courses.json";
import modules from "./modules.json";
import assignments from "./assignments.json";
export {
  courses, modules, assignments, };
```

Use the ***useParams()*** hook to retrieve the ***course***'s ID and then find all the assignments for that course from the database's ***assignments*** array. Render the assignments as links that encode the ***course***'s ID and the ***assignment***'s ID in the URL's path. Use the ***assignment***'s ***ID*** to render the corresponding assignment in the ***AssignmentEditor*** page. Modify ***assignments.json*** as needed. The assignments screen should render similar to Figure 3.10.8.

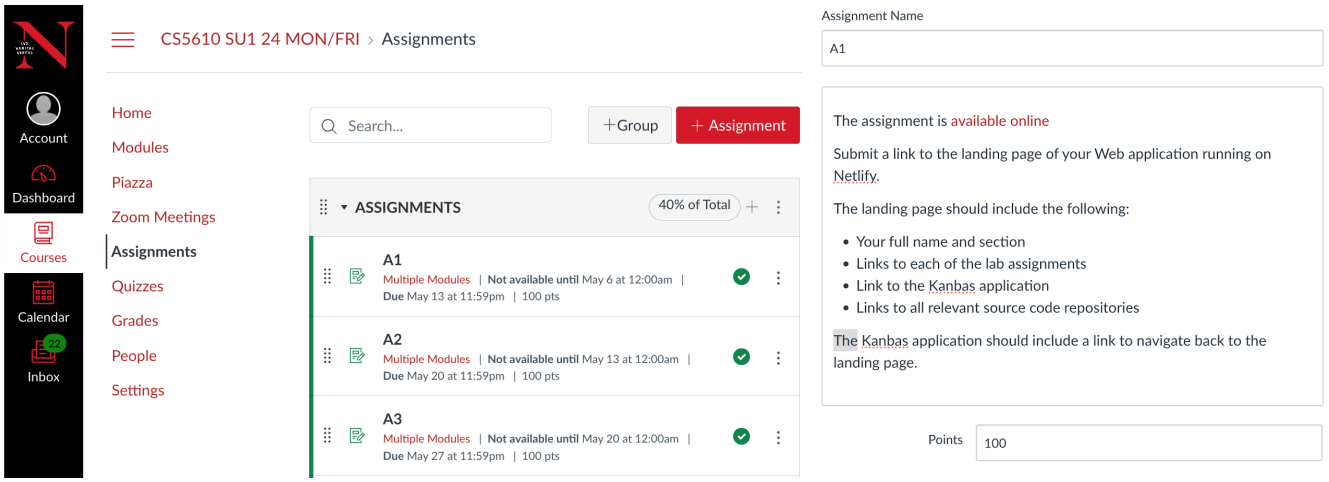


Figure 3.10.8 - Data Driven Assignments Screen

Figure 3.10.8.1 - Data Driven Assignment Editor Screen

3.10.8.1 Data Driven Assignment Editor Screen (On Your Own)

Based on the ***AssignmentEditor*** page implemented in earlier chapters, refactor the screen so that it renders the information for the assignment selected in the ***Assignments*** screen. Use ***useParams()*** to parse the ***assignment***'s ***ID***s from the URL and retrieve the assignment from the database's ***assignments*** object. Display the ***title*** of the selected assignment as well as the ***description***, ***points***, ***due date***, and ***available date***. Use ***useParams()*** to parse the ***course***'s ***ID*** from the URL and implement the ***Cancel*** and ***Save*** buttons as ***Link*** so that clicking either one navigates back to the ***Assignments*** screen for the current course. The ***Assignment Editor*** screen should look as shown in Figure 3.10.8.1. Other fields such as ***Submission Type*** and ***Assignment Group*** are not shown for brevity, but they should still appear as implemented in earlier chapters.

3.10.9 Data Driven People Screen

Previous chapters implemented and styled the ***People*** screen that displays a list of users as a table. The current implementation is static, always showing the same hard coded list of students. In this section, refactor the ***People*** screen to display students, teaching assistants, and faculty associated with the selected course. Use the data provided as an example list of users and save the file as ***users.json*** file in the ***Database*** folder. Import ***users.json*** into ***Database/index.ts*** so that it can be imported from the ***People/Table/page.tsx***. Also create an ***enrollments.json*** file that contains enrollment data establishing which students are enrolled in which course as shown below.

app/(kambaz)/database/enrollments.json

```
[
  { "_id": "1", "user": "123", "course": "RS101" },  { "_id": "2", "user": "234", "course": "RS101" },
  { "_id": "3", "user": "345", "course": "RS101" },  { "_id": "4", "user": "456", "course": "RS101" },
  { "_id": "5", "user": "567", "course": "RS101" },  { "_id": "6", "user": "234", "course": "RS102" },
  { "_id": "7", "user": "789", "course": "RS102" },  { "_id": "8", "user": "890", "course": "RS102" },
  { "_id": "9", "user": "123", "course": "RS102" } ]
```

In the ***PeopleTable*** screen, parse the current course ID from the path using ***useParams*** so that the users associated with the course can be filtered based on their enrollment.

app/(kambaz)/courses/[cid]/people/table/page.tsx

```
"use client"
import React from "react";
import * as db from "../../../../database";
import { useParams } from "next/navigation";
export default function PeopleTable() {
  const { cid } = useParams();
  const { users, enrollments } = db;
  return (
    <div id="wd-people-table"> ...  </div> );}
```

Replace the hardcoded rows displaying the users with an expression that displays only the users related to the currently selected course. The ***users.filter*** expression below filters the array of users in the database. The filter function searches the users in the ***enrollments*** array based on the user's ID and the user's enrollment in the selected course. Navigate to various courses and confirm that only the users enrolled in the courses are shown in the ***People*** screen.

app/(kambaz)/courses/[cid]/people/table/page.tsx

```
<tbody>
  {users
    .filter((usr) =>
      enrollments.some((enrollment) => enrollment.user === usr._id && enrollment.course === cid)
    )
    .map((user: any) => (
      <tr key={user._id}>
        <td className="wd-full-name text-nowrap">
          <FaUserCircle className="me-2 fs-1 text-secondary" />
          <span className="wd-first-name">{user.firstName}</span>
          <span className="wd-last-name">{user.lastName}</span>
        </td>
        <td className="wd-login-id">{user.loginId}</td>
        <td className="wd-section">{user.section}</td>
        <td className="wd-role">{user.role}</td>
        <td className="wd-last-activity">{user.lastActivity}</td>
        <td className="wd-total-activity">{user.totalActivity}</td>
      </tr>
    ))}
</tbody>
```

3.11 Deliverables

1. In the same React application created in earlier chapters, ***kambaz-next-js***, complete all the exercises described in this document
2. In a branch called ***a3***, add, commit and push the source code of the React application ***kambaz-next-js*** to the same remote source repository in ***GitHub.com*** created in an earlier chapter. Here's an example of how to add, commit and push your code

```
$ git checkout -b a3
$ git add .
$ git commit -am "a3 JavaScript"
$ git push
```

3. Deploy the ***a3*** branch to a new ***Vercel*** project and include the public URL to your project in your submission.
4. Make sure ***labs/page.tsx*** contains a ***TOC.tsx*** that references each of the labs and ***Kambaz***. Add a link to your repository in GitHub. The link should have an ***ID*** attribute with a value of ***wd-github***.
5. In ***labs/page.tsx***, add your full name: first name first and last name second. Use the same name as in Canvas.
6. Deploy the branch for this chapter to your ***kambaz-next-js*** React application to ***Vercel*** as described.
7. As a deliverable in ***Canvas***, submit the URL to the ***a3*** branch deployment of your React application running on Vercel.